

Relatório 7 - Prática: Mergulho nas IDEs Online para Aprendizado de Máquina (I)

Gustavo Nunes da Silva

1. Introdução

O objetivo desse card foi conhecer na prática alguns ambientes que são usados pra trabalhar com Python em ciência de dados. Eu segui o caminho mostrado no vídeo começando pelo terminal e depois passando pelos notebooks até chegar nos ambientes que ficam na nuvem. A ideia foi usar o mesmo código nos cinco ambientes e ir vendo o que mudava em cada um e o que cada ferramenta acabava facilitando.

2. Desenvolvimento

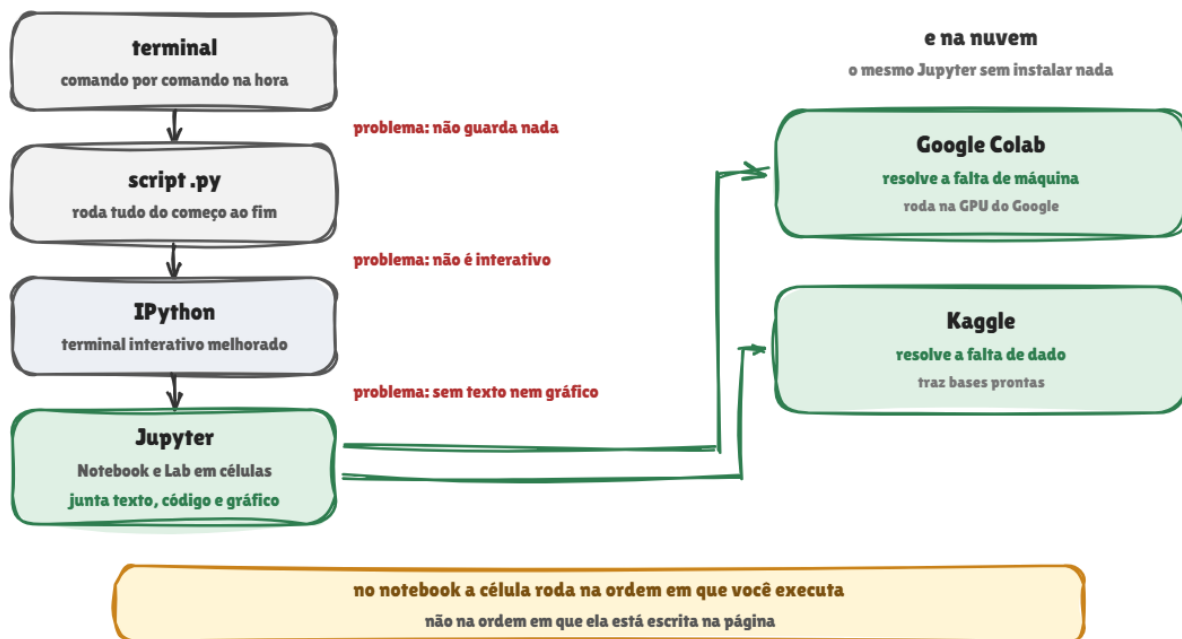
Então fazendo os cinco ambientes na ordem do vídeo eu percebi que eles não são exatamente ferramentas concorrentes. Na verdade, cada um vai resolvendo algum problema que aparece no anterior. Foi isso que eu tentei mostrar na Figura 1. Para conseguir comparar eu usei o mesmo gráfico de seno nos ambientes e só fui mudando a ferramenta usada.

Figura 1

O que cada ambiente resolve

cada ambiente resolve um problema do anterior

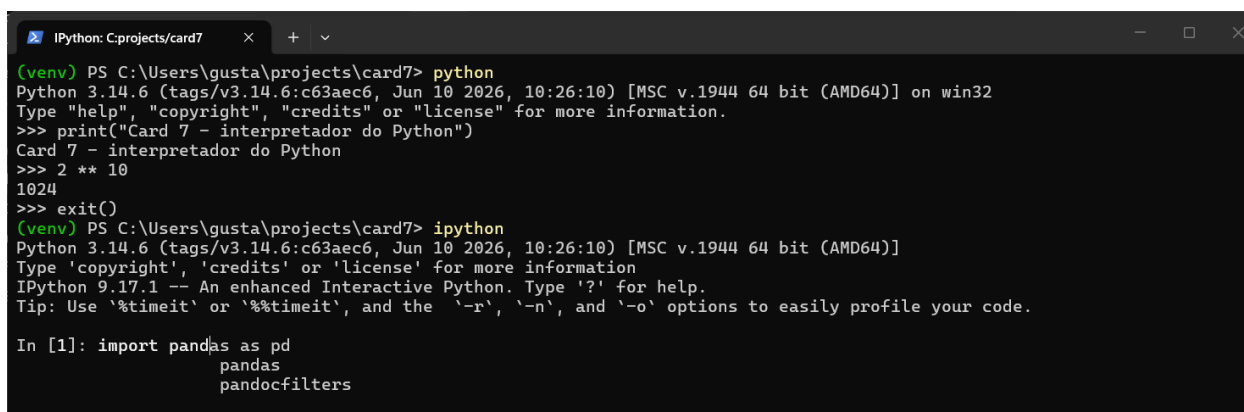
do terminal até o Jupyter e depois para a nuvem



Fonte – Autoria própria (elaborado no Excalidraw).

Eu comecei pelo Python no terminal usando o comando python. É a forma mais simples porque eu digito uma coisa e já vejo o resultado na hora então achei bom para fazer testes rápidos como uma conta ou um print. O problema é que quando o código fica maior começa a ficar bem ruim de organizar. Depois tem o script que já deixa o código salvo em um arquivo e isso ajuda bastante, mas sempre que eu faço uma alteração preciso rodar o arquivo inteiro de novo. Também testei o IPython que deixa o terminal bem mais completo com destaque no código e sugestões quando uso o Tab. Na Figura 2 dá pra ver as sugestões que aparecem quando eu começo a digitar import pand.

Figura 2



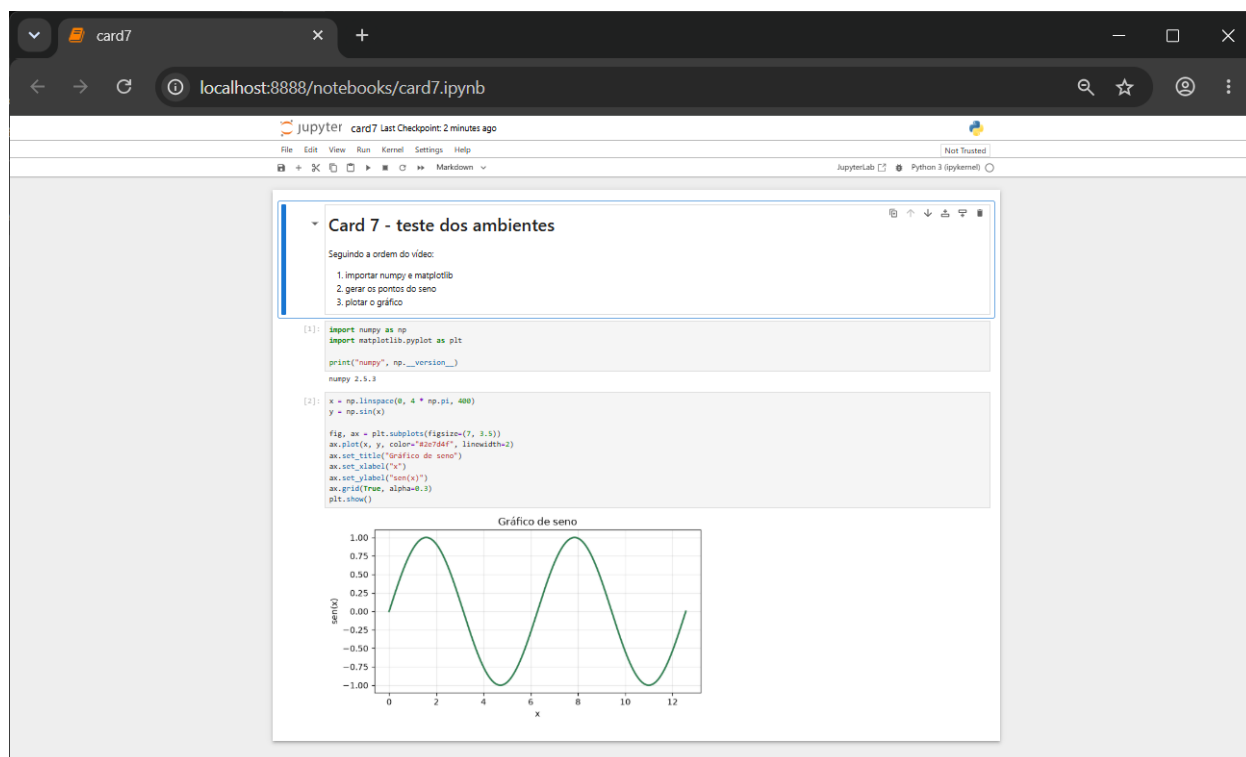
```
(venv) PS C:\Users\gusta\projects\card7> python
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Card 7 - interpretador do Python")
Card 7 - interpretador do Python
>>> 2 ** 10
1024
>>> exit()
(venv) PS C:\Users\gusta\projects\card7> ipython
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.17.1 -- An enhanced Interactive Python. Type '?' for help.
Tip: Use '%timeit' or '%timeit', and the '-r', '-n', and '-o' options to easily profile your code.

In [1]: import pandas as pd
        pandas
        pandocfilters
```

Fonte – Autoria própria (captura de tela).

Depois eu fui para o Jupyter Notebook e pra mim foi aí que começou a ficar bem mais interessante. Quando eu uso o comando jupyter notebook ele abre um servidor local na porta 8888 e mostra tudo no navegador, o que achei interessante nele é que parece um site, mas na verdade está rodando na própria máquina e isso aparece pelo localhost na Figura 3. Dentro dele eu consigo colocar células de código e células de markdown então dá pra deixar título texto código e resultado tudo no mesmo lugar. Eu fiz um título uma lista e depois coloquei o código do gráfico e o resultado apareceu logo abaixo. Para mim essa foi uma diferença bem grande em relação ao script porque fica muito mais fácil acompanhar o que eu estou fazendo.

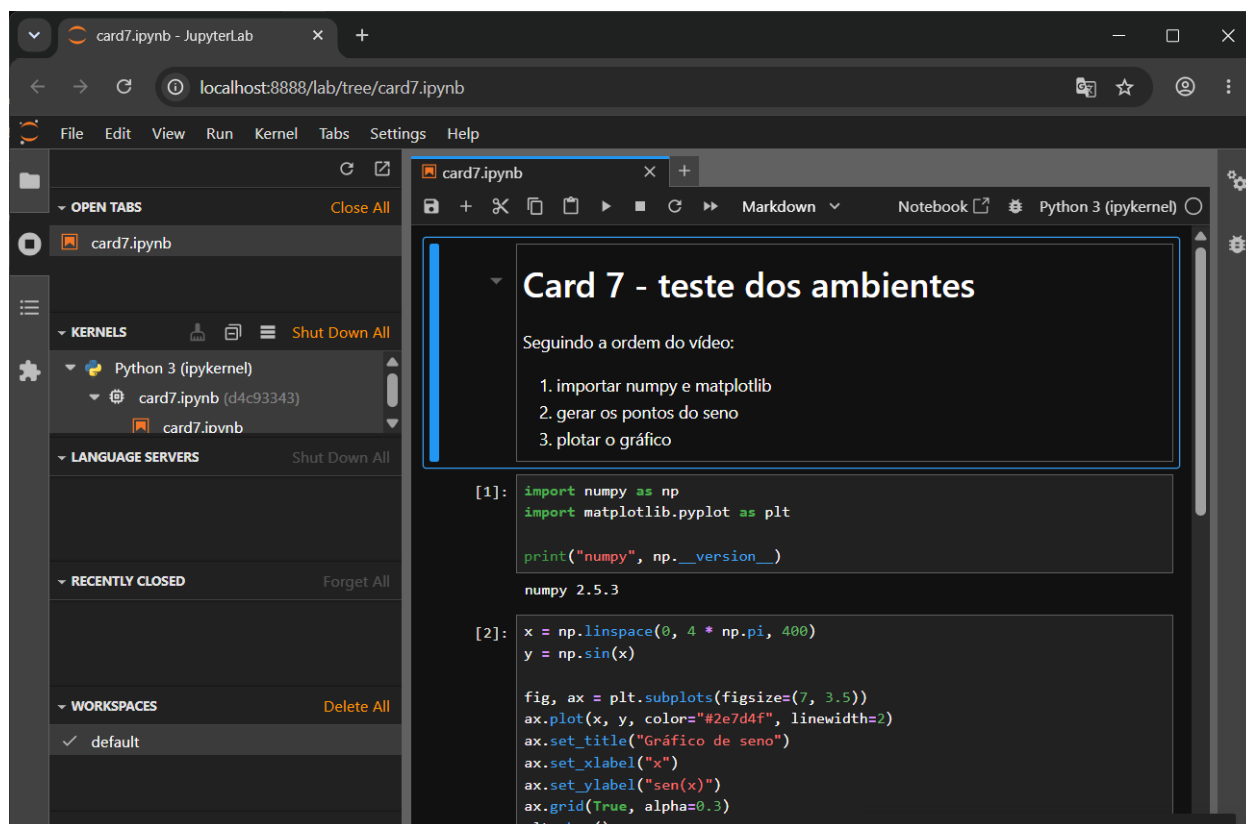
Figura 3



Fonte – Autoria própria (captura de tela).

Depois testei o Jupyter Lab que é uma versão mais completa do Jupyter. Em vez de parecer só um documento ele já funciona mais como um ambiente de trabalho com painel lateral abas e os kernels que estão rodando como aparece na Figura 4. Eu também achei legal poder deixar as abas lado a lado e trocar o tema. Outra coisa que achei útil foi poder exportar o notebook pra HTML ou PDF porque assim dá pra mandar a análise pra alguém sem precisar que a pessoa tenha o Jupyter instalado.

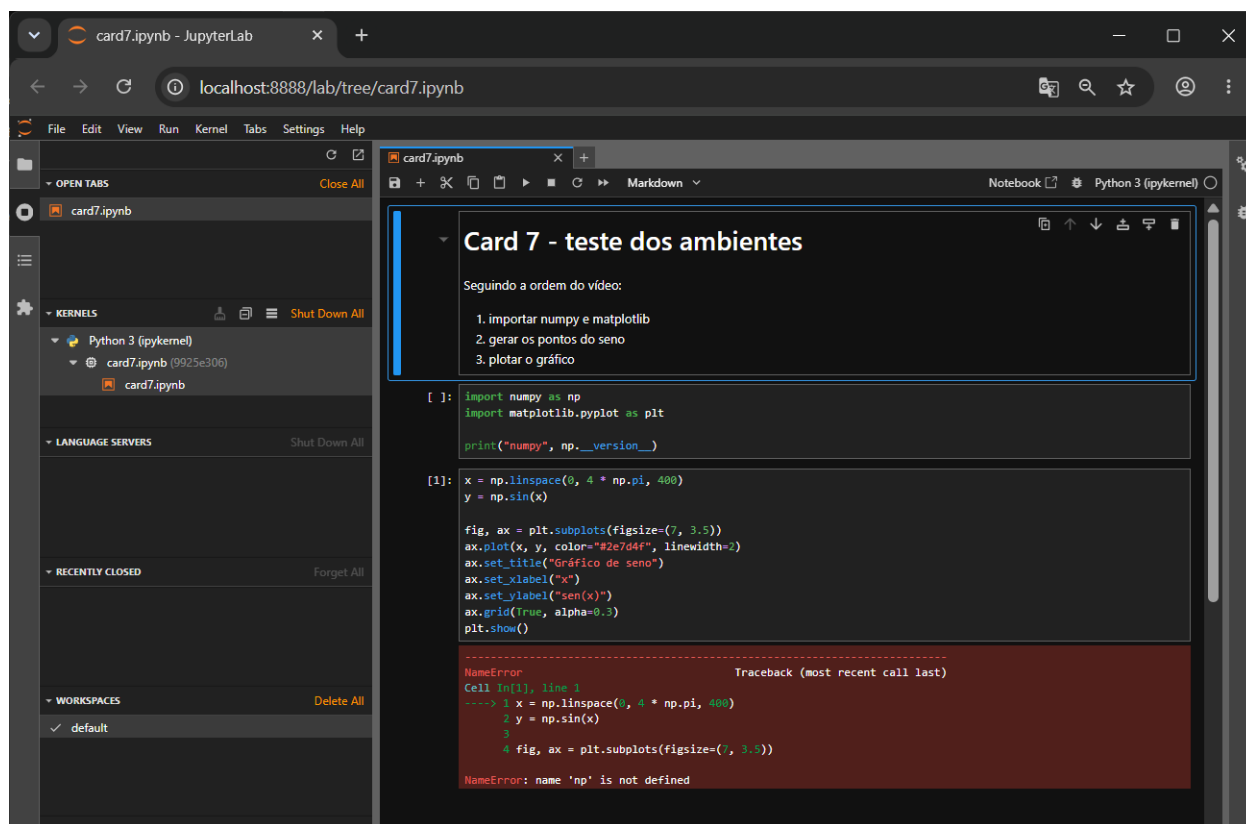
Figura 4



Fonte – Autoria própria (captura de tela).

Uma coisa que eu achei bem importante nessa parte foi perceber que as células do Jupyter não precisam ser executadas na mesma ordem em que aparecem. Eu fiz um teste de propósito rodando primeiro a célula do gráfico antes da parte que importa as bibliotecas. Aí deu erro porque o np ainda não existia como mostra a Figura 5. Também entendi que o número entre colchetes mostra a ordem em que a célula foi executada e não a posição dela no notebook. Então dá pra ter um notebook que parece estar certo olhando a tela mas que na prática depende da ordem em que eu fui executando as células.

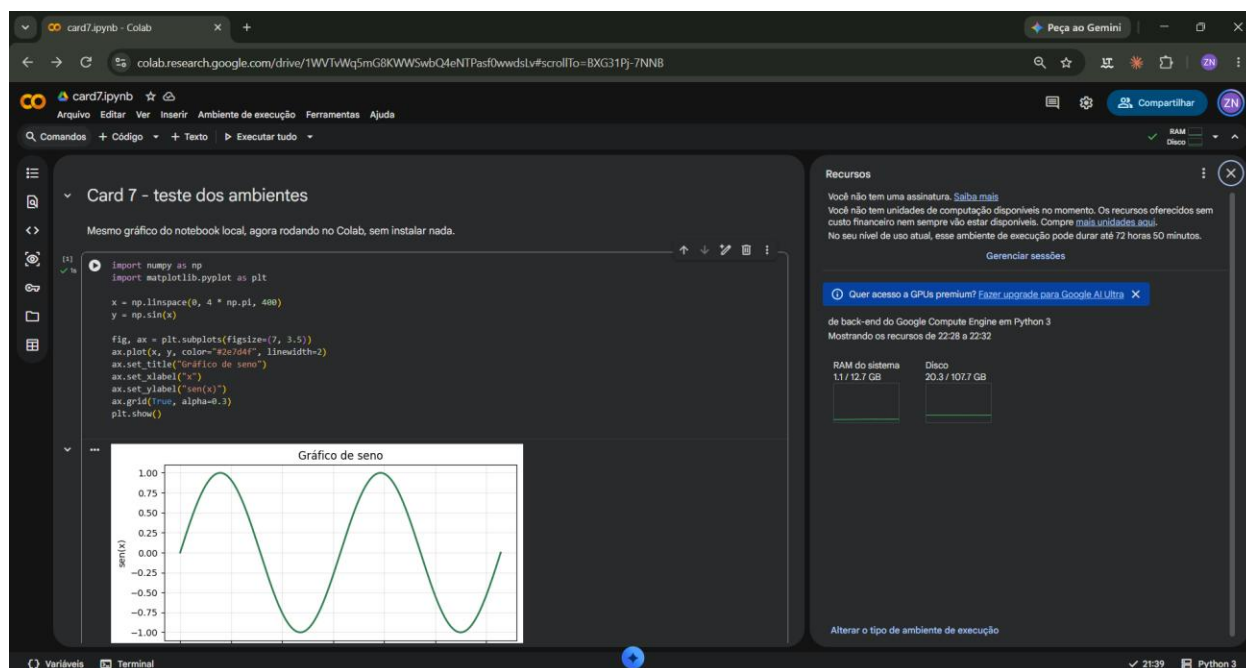
Figura 5



Fonte – Autoria própria (captura de tela).

Depois eu passei para os ambientes na nuvem começando pelo Google Colab. O que eu mais gostei nele foi não precisar instalar as bibliotecas porque várias coisas de ciência de dados já vêm prontas e o código roda direto no navegador. Na Figura 6 aparece o painel de recursos onde eu consigo ver a memória e o espaço que a sessão está usando e algumas informações sobre o tempo que aquele ambiente pode ficar disponível. Também achei interessante poder escolher uma GPU no ambiente de execução porque isso pode ajudar bastante quando a máquina que eu estou usando não tem uma placa desse tipo.

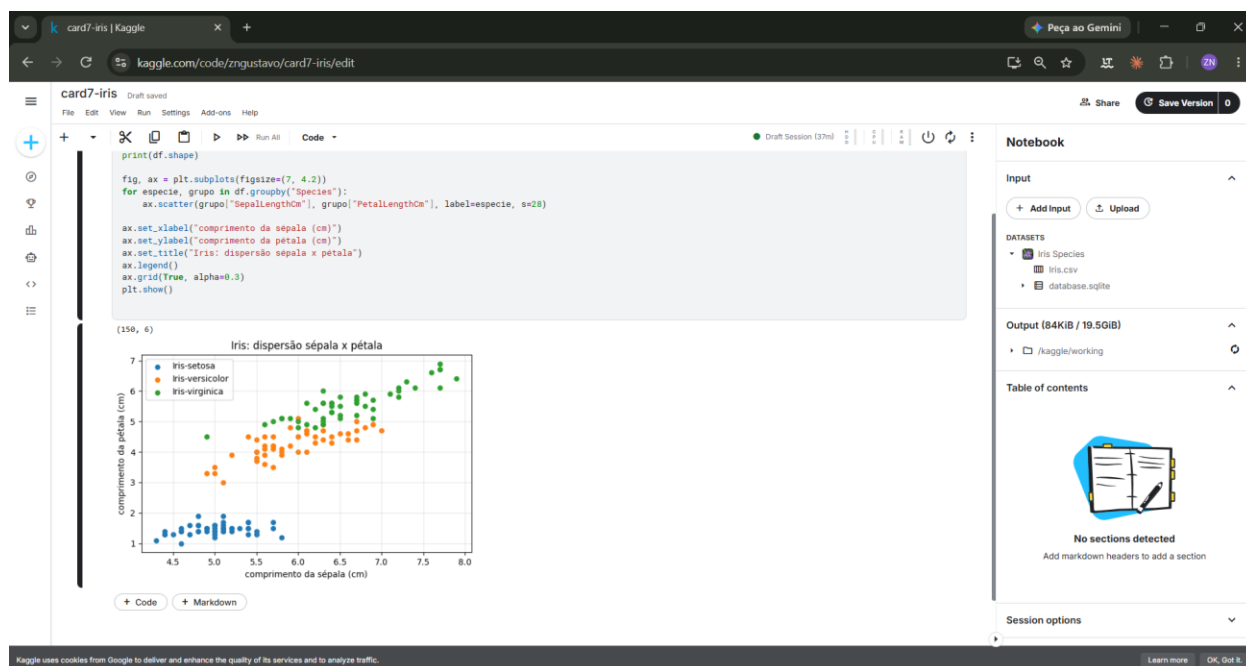
Figura 6



Fonte – Autoria própria (captura de tela).

Por último eu testei o Kaggle e achei interessante principalmente pela facilidade de trabalhar com os dados. No próprio notebook eu consigo adicionar uma base pública pelo painel lateral sem precisar baixar o arquivo e depois ficar enviando ele de novo. Eu carreguei a base Iris e fiz um gráfico comparando o comprimento da sépala com o da pétala e separando as três espécies como aparece na Figura 7. Uma coisa que chamou bastante minha atenção foi que pra salvar uma versão o notebook precisa rodar inteiro sem erro. Isso acaba evitando um problema parecido com o que eu mostrei antes no Jupyter Lab porque lá eu consigo executar as células fora de ordem e deixar coisas funcionando só por causa da ordem em que eu rodei. No Kaggle essa regra ajuda a garantir que o notebook consiga rodar do começo ao fim.

Figura 7.



Fonte – Autoria própria (captura de tela).

No final eu também testei alguns atalhos que ajudam bastante nesses ambientes de células. Shift e Enter executa a célula e já vai pra próxima enquanto Control e Enter executa sem sair da célula atual. Com Esc eu entro no modo de comando e aí posso usar A e B pra criar células acima ou abaixo e apertar D duas vezes pra apagar. Também dá pra usar M e Y pra trocar entre markdown e código. Eu comecei a usar bastante Esc M e Esc Y porque no começo é bem fácil escrever código numa célula de markdown e ficar sem entender por que aquilo não executa.

3. Conclusão

Depois de fazer tudo isso eu consegui entender melhor pra que serve cada ambiente. O terminal é mais pra testar coisas rápidas o Jupyter deixa o código e a explicação juntos e os ambientes na nuvem ajudam quando eu não quero depender tanto da máquina que estou usando ou quando já quero trabalhar com dados que estão online. O que mais ficou pra mim foi o cuidado com a ordem das células porque dá pra fazer um notebook funcionar durante os testes e depois outra pessoa abrir e ele dar erro. Então antes de entregar eu acho importante reiniciar o kernel e rodar tudo de novo desde o começo pra ter certeza de que está funcionando.

4. Referências

MULLA, Rob. Jupyter Notebook Complete Beginner Guide - From Jupyter to Jupyterlab, Google Colab and Kaggle!. YouTube, 2023. Disponível em: https://www.youtube.com/watch?v=5pf0_bpNbkw.